

Payment Gateway System

Java program - Polymorphism (Overloading & Overriding)

Program 3: Payment Gateway (Method Overloading & Overriding)

Demonstrates compile-time polymorphism (overloaded makePayment methods) and runtime polymorphism (overridden makePayment across CreditCardPayment, UPIPayment, NetBankingPayment), plus edge-case handling.

Source Code

```
class Payment {
    Payment() {
        System.out.println("Payment object created.");
    }
    void makePayment(double amount) {

        if (amount <= 0) {
            System.out.println("Invalid payment amount.");
            return;
        }
        System.out.printf("Payment of ₹%.2f processed successfully.%n", amount);
    }
    void makePayment(double amount, String transactionId) {
        if (amount <= 0) {
            System.out.println("Invalid payment amount.");
            return;
        }
        if (transactionId == null || transactionId.isEmpty()) {
            System.out.println("Invalid transaction ID.");
            return;
        }
        if (!transactionId.matches("TXN\\d+")) {
            System.out.println("Invalid transaction ID format.");
            return;
        }

        System.out.printf(
            "Payment of ₹%.2f processed successfully with Transaction ID: %s%n",
            amount, transactionId
        );
    }
}
class CreditCardPayment extends Payment {
    CreditCardPayment() {
        System.out.println("CreditCardPayment object created.");
    }
    @Override
    void makePayment(double amount) {

        if (amount <= 0) {
            System.out.println("Invalid payment amount for Credit Card.");
            return;
        }

        System.out.printf(
            "Credit Card Payment of ₹%.2f processed successfully.%n",
            amount
        );
    }
}
class UPIPayment extends Payment {
    UPIPayment() {
        System.out.println("UPIPayment object created.");
    }

    @Override
    void makePayment(double amount) {

        if (amount <= 0) {
            System.out.println("Invalid payment amount for UPI.");
        }
    }
}
```

```

        return;
    }

    System.out.printf(
        "UPI Payment of ₹%.2f processed successfully.%n",
        amount
    );
}
}

class NetBankingPayment extends Payment {

    NetBankingPayment() {
        System.out.println("NetBankingPayment object created.");
    }

    @Override
    void makePayment(double amount) {

        if (amount <= 0) {
            System.out.println("Invalid payment amount for Net Banking.");
            return;
        }

        System.out.printf(
            "Net Banking Payment of ₹%.2f processed successfully.%n",
            amount
        );
    }
}

public class Main {

    public static void main(String[] args) {

        System.out.println("===== PAYMENT GATEWAY =====\n");
        Payment payment;

        System.out.println("Test Case 1: Credit Card Payment");

        payment = new CreditCardPayment();
        payment.makePayment(5000.00);

        System.out.println("\nTest Case 2: UPI Payment");

        payment = new UPIPayment();
        payment.makePayment(1200.50);

        System.out.println("\nTest Case 3: Net Banking Payment");

        payment = new NetBankingPayment();
        payment.makePayment(8500.00);
        System.out.println("\nTest Case 4: Compile-Time Polymorphism");

        payment = new Payment();
        payment.makePayment(2500.00, "TXN1001");

        System.out.println("\nTest Case 5: Runtime Polymorphism");

        payment = new CreditCardPayment();
        payment.makePayment(1000.00);

        payment = new UPIPayment();
        payment.makePayment(1500.00);

        payment = new NetBankingPayment();
        payment.makePayment(2000.00);

        System.out.println("\nTest Case 6: Negative Payment Amount");

        payment = new CreditCardPayment();
        payment.makePayment(-1000.00);
        System.out.println("\nTest Case 7: Zero Payment Amount");
    }
}

```

```

        payment = new UPIPayment();
        payment.makePayment(0.00);

        // TEST CASE 8 - EMPTY TRANSACTION ID

        System.out.println("\nTest Case 8: Empty Transaction ID");

        payment = new Payment();
        payment.makePayment(3000.00, "");

        System.out.println("\nTest Case 9: Null Payment Object");

        Payment nullPayment = null;

        try {
            nullPayment.makePayment(1000.00);
        }
        catch (NullPointerException e) {
            System.out.println("Error: Payment object is null.");
        }

        System.out.println("\nTest Case 10: Invalid Transaction ID Format");

        payment = new Payment();
        payment.makePayment(4500.00, "123");

    }
}

```

Output

===== PAYMENT GATEWAY =====

Test Case 1: Credit Card Payment
 Payment object created.
 CreditCardPayment object created.
 Credit Card Payment of ₹5000.00 processed successfully.

Test Case 2: UPI Payment
 Payment object created.
 UPIPayment object created.
 UPI Payment of ₹1200.50 processed successfully.

Test Case 3: Net Banking Payment
 Payment object created.
 NetBankingPayment object created.
 Net Banking Payment of ₹8500.00 processed successfully.

Test Case 4: Compile-Time Polymorphism
 Payment object created.
 Payment of ₹2500.00 processed successfully with Transaction ID: TXN1001

Test Case 5: Runtime Polymorphism
 Payment object created.
 CreditCardPayment object created.
 Credit Card Payment of ₹1000.00 processed successfully.
 Payment object created.
 UPIPayment object created.
 UPI Payment of ₹1500.00 processed successfully.

Payment object created.
NetBankingPayment object created.
Net Banking Payment of ₹2000.00 processed successfully.

Test Case 6: Negative Payment Amount
Payment object created.
CreditCardPayment object created.
Invalid payment amount for Credit Card.

Test Case 7: Zero Payment Amount
Payment object created.
UPIPayment object created.
Invalid payment amount for UPI.

Test Case 8: Empty Transaction ID
Payment object created.
Invalid transaction ID.

Test Case 9: Null Payment Object
Error: Payment object is null.

Test Case 10: Invalid Transaction ID Format
Payment object created.
Invalid transaction ID format.